

Лабораторная работа №2

Блочно - симметричные шифры.

2.1. Цель лабораторной работы

Цель лабораторной работы – получение теоретических и практических навыков по обеспечению конфиденциальности и целостности информации с помощью блочных симметричных шифров; знакомство с архитектурой таких шифров и их параметрами; получение практических навыков использования различных режимов шифрования.

2.2. Рекомендации к подготовке выполнения ЛР

Необходимо ознакомиться с основами симметричного шифрования, принципами построения симметричных блоков шифров, преимуществами и недостатками разных режимов использования таких шифров.

2.3. Общие положения ЛР

Режим шифрования

Под режимом шифрования будем понимать такой алгоритм применения блочного шифра, который позволяет преобразовать открытый текст, длина которого превышает один блок, в шифртекст;

после передачи шифртекста открытым каналом однозначно возобновить открытый текст;

повысить стойкость к разным видам атак;

реализовать дополнительные услуги безопасности.

При таком использовании собственно блочный шифр является составляющей частью алгоритма применения, поскольку он работает только с отдельным блоком данных, а используется для шифрования целого сообщения, которое состоит из некоторого числа n блоков. Больше того, длина сообщения может не быть кратной длине блока, то есть его невозможно разбить на целое число таких блоков.

Режимы шифрования:

ECB (Electronic Code Book) – электронная кодовая книга;

CBC (Cipher Block Chaining) - сцепления блоков шифротекста;

CFB (Cipher Feedback) – обратная загрузка шифротекста;

OFB (Output Feedback) – обратная загрузка выходных данных.

Электронная кодовая книга - ECB (Electronic Code Book)

В этом режиме шифрования/ дешифрования i -го блока открытого текста / шифротекста выполняется независимо от других блоков:

$$c_i = E_k(m_i), \quad m_i = D_k(c_i).$$

Недостатком данного режима является то, что одинаковые блоки выходного текста будут преобразовываться в одинаковые блоки шифротекста, которые дают возможность злоумышленнику, во – первых, делать предположения про характер информации в открытом тексте, а, во – вторых, подменить один или несколько блоков шифротекста. Преимущество режима можно назвать простоту реализации, а также возможность распараллеливания процедуры шифрования.

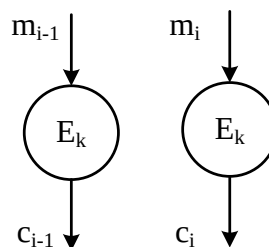


Схема режима приведена на рис. 2. 1.

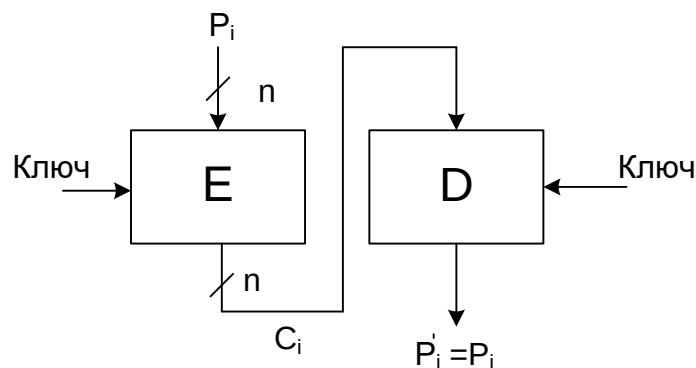


Рис. 2. 1. Режим электронной кодовой книги.

На выход алгоритма находят k -битовые блоки открытого текста $P = p_1, p_2, \dots, p_i, \dots$. На выходе формируются n -битовые блоки шифрования $C = c_1, c_2, \dots, c_i, \dots$.

Шифрование: $c_i = E_k(p_i)$.

Дешифрование: $p_i = D_k(c_i)$.

Обозначим следующие свойства ECB режима:

1. Блочные шифры в таком режиме не обеспечивают сокрытия шаблонов данных – идентичные блоки открытого текста (при том же самом ключе) преобразовываются в идентичный шифротекст.

2. Блоки шифрования независимо друг от друга. Реорганизация блоков шифротекста приведёт к реорганизации блоков открытого текста. Поскольку блоки шифротекста независимы, умышленная подстановка ЕВС – блоков (например, вставка блока, который часто встречается) не влияет на дешифрование смежных блоков.

3. Один и более ошибочных битов в одном блоке шифротекста влияют на дешифрование только этого блока. Результат дешифрования блока с ошибками является случайной величиной, можно восстановить около 50 % битов открытого текста.

4. Существенным недостатком ЕВС является то, что одинаковые блоки открытого текста на одинаковом ключе всегда имеют одинаковый зашифрованный вид. По этому для больших сообщений ЕВС – режим считается опасным. Если сообщение имеет много одинаковых блоков, то вовремя криптоанализа эту закономерность будет определено.

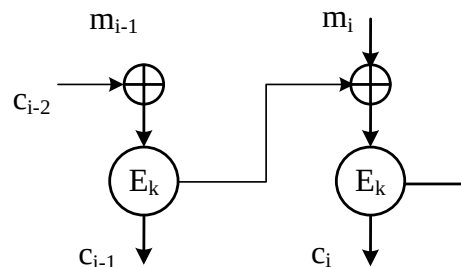
Перечисленные особенности ограничивают использование режима ЕВС: его не рекомендуют применять для шифрования сообщений, длиной больше одного блока, либо если повторно используются ключи для шифрования больше, чем одного одноблочного сообщения. В этом случае секретность ключа можно обеспечить добавлением к каждому блоку случайных битов.

Сцепления блоков шифротекста CBC (Cipher Block Chaining)

Режим CBC допускает такие алгоритмы шифрования/дешифрования:

$$c_i = E_k(m_i \oplus c_{i-1}), \quad m_i = D_k(c_i) \oplus c_{i-1}.$$

В режиме CBC каждый блок открытого текста добавляется к блокам шифротекста, полученных на предыдущем этапе. Таким образом, происходит сцепление блоков один за другим и независимая манипуляция с каждым из них невозможна, а одинаковые блоки будут давать на выходе разные результаты. Однако, при этом усложняется распараллеливание процедуры расшифрования.



Режим сцепления блоков шифра включает использование n – разрядного вектора инициализации IV (Initializing Vector). В схему добавляется регистр или буфер обратной связи, куда сначала записывается значение IV, а потом – следующие значения блоков шифротекста. На рис. 2.2 приведена схема CBC. Для получения первого блока зашифрованного сообщения используется вектор инициализации (IV), для которого выполняется операция XOR с первым блоком нешифрованного сообщения при расшифровании для IV выполняется операция XOR с выходом алгоритма расшифрования для получения первого блока нешифрованного текста.

IV должен быть известен как и отправителю, так и получателю. Для максимальной безопасности IV должен быть защищен так же само, как и ключ.

На выход алгоритма находит k – битовый ключ K , n – битовый IV, последовательность n – битовых блоков открытого текста $P = p_1, p_2, \dots, p_i, \dots, p_t$.

Последовательность блоков шифрования $C = c_1, c_2, \dots, c_t$ вычисляется по правилу: $c_i = E_k(p_i \oplus c_{i-1})$; $i = 1, \dots, t$; $c_0 = IV$.

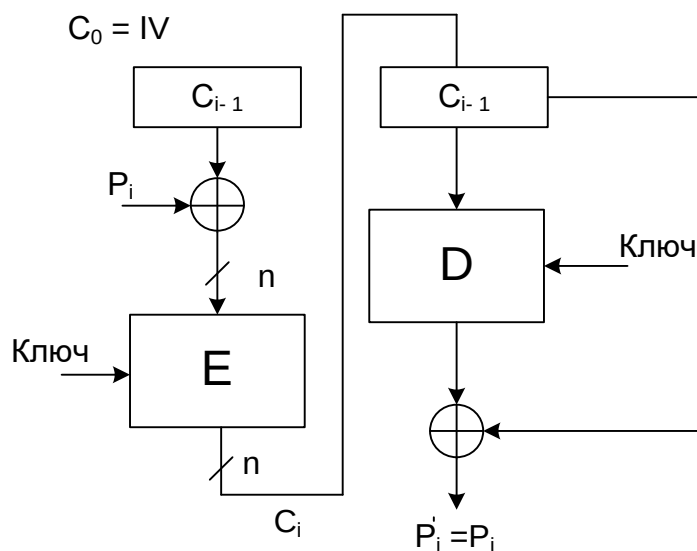


Рис. 2.2. Режим сцепления блоков шифра.

Расшифрование осуществляется так: $p_i = c_{i-1} \oplus D_k(c_i)$; $i = 1, \dots, t$; $c_0 = IV$.

Свойства режима:

1. Идентичные блоки шифрования шифротекста формируются тогда, когда шифруется тот самый открытый текст на тех самых же ключах и том же самом векторе инициализации. Замена вектора инициализации, ключа или первого блока сообщения (например, используя счетчик или случайную величину) приводит к разным шифротекстам.

2. Механизм сцепления приводит к тому, что шифротекст c_i зависит от p_i и всех предыдущих блоков открытого текста. В этом случае зависимость от предыдущих блоков содержится в значении предыдущего блока шифротекста. Следовательно, замена порядка блоков шифротекста влияет на расшифрование. Для того, чтобы правильно расшифровать блок шифротекста, необходимо иметь правильный блок шифротекста.

3. Единичная блокавая ошибка в блоке шифротекста c_i влияет на дешифрование как минимум двух блоков c_i и c_{i+1} (поскольку p_i зависит от c_i и c_{i+1}). Блок p'_i восстановленный из c_i конечно является полностью случайным (50% битов ошибочны), и в то время как открытый текст p'_{i+1} будет иметь неправильных битов ровно столько, сколько из имеет блок c_i . Таким образом, злоумышленник может вызвать предвиденные изменения битов в p'_{i+1} изменяя соответственные биты блока c_i .

4. CBC – режим является реализацией шифрования с авто ключом за шифротекстом в том понимании, что если есть ошибка в блоке c_i (включая потерю одного или более выходных блоков), но нет ошибок в c_{i+1} , c_{i+2} , то возможно конкретное дешифрования p_{i+2} .

3 точки зрения практического использования блочных шифров в данном режиме необходимо также учитывать следующие рекомендации:

1. Распространение ошибок при шифровании. Хотя расшифрование в CBC – режиме позволяет восстановить открытый текст с ошибок в блоках шифротекста, модификация блока открытого текста p_i вовремя шифрования изменяет все следующие блоки шифротекста. Это усложняет (если не исключает) использование режимов сцепления когда необходим случайный доступ (считывания/записи) к зашифрованным данным. Альтернативой является режим ECB.

2. Самосинхронизация и группирование ошибок. Не обращая внимания на то, что самосинхронизация возможна при наличии ошибок в битах, при «потерянных» битах, что является причиной ошибки определения меж блока (ошибка целостного кадра), она невозможна не в CBC – режиме, не в других режимах.

3. Целостность IV в CBC. Не обращая внимания на то, что вектор инициализации IV в CBC – режиме не секретный, должна быть обеспечена его целостность, поскольку умышленная модификация IV позволяет злоумышленнику выполнять предвиденные замены битов в расшифрованном первом блоке открытого текста. Использование секретных IV является одним из методов предотвращения таких модификаций. Однако, если нужно обеспечить целостность сообщения, должны использоваться другие механизмы, потому что механизмы шифрования, обычно, гарантируют только конфиденциальность.

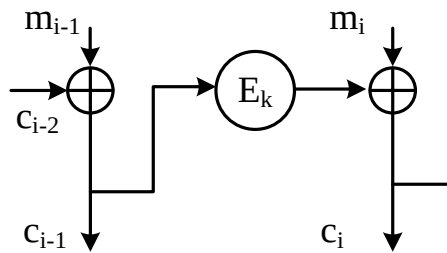
Обратная загрузка шифротекста - CFB (Cipher Feedback)

В режиме CFB также происходит «маскирование» блока открытого текста уже зашифрованными блоками:

$$c_i = E_k(m_i \oplus c_{i-1}), \quad m_i = D_k(c_i) \oplus c_i.$$

По своим возможностям этот режим похож на режим CBC, но если длина сообщения не кратная размеру блока шифра, то в режиме CBC необходимо дополнять последний блок дополнительными битами и сообщать на принимающую сторону настоящий размер сообщения, а

режим CFB позволяет сформировать шифротекст того же размера, что и выходное сообщение.



В CFB – режиме на вход подается k – битовый ключ K , n – битовый IV , последовательность r – битовых блоков открытого текста $P = p_1, p_2, \dots, p_u$, где $1 \leq r \leq n$. На выходе формируются r – битовые блоки шифротекста $C = c_1, c_2, \dots, c_u$.

Шифрование осуществляется так. Сначала в буфер обратной связи, записывается значение $IV - I_1 = IV$. Потом для всех $1 \leq i \leq u$ выполняются следующие операции:

1. Вычисляются функции шифрования

$$O_i = E_k(I_i).$$

2. Выбираются r левых битов результата шифрования O_i

$$t_i = O_i \sim r,$$

где $O_i \sim r$ означает операция выборки.

3. Формирование и передача r – битового блока шифротекста c_i .

$$c_i = p_i \oplus t_i.$$

4. Формирование переменной обратной связи с блока шифротекста, согласно выражению

$$I_{i+1} = 2^r I_i \oplus c_i \bmod 2^n$$

Эта операция эквивалентна смещению значения буферного регистра I_i на r – битов вправо и записи в конец регистра r – битового блока c_i . Расшифрование выполняется соответственно:

$$P_i = c_i \oplus t_i; \quad i = 1, \dots, u,$$

где t_i , O_i и I_i вычисляются соответственно выражению, приведенного выше и:

$$I_1 = IV.$$

На рис. 2.3 приведена схема режима CFB.

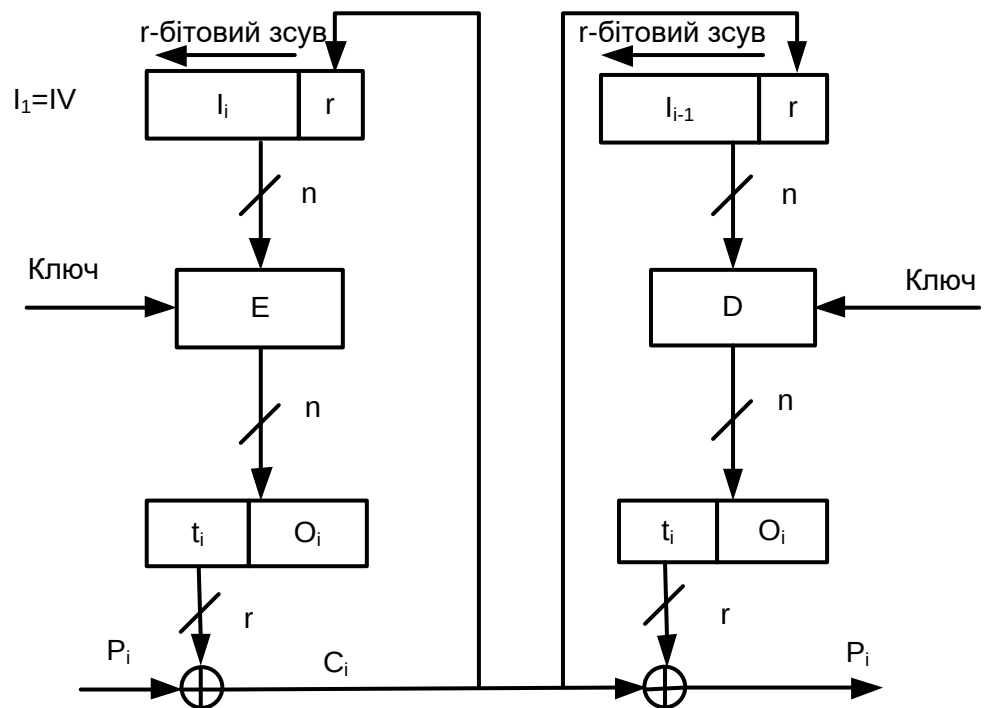


Рис. 2.3. Режим обратной связи по шифротексту.

Отметим, что фактически необходимо иметь еще и буфер обратной связи для хранения c_{i+1} , которая конечно же равняется n .

CFB – режим имеет следующие возможности:

1. Как и в CBC – режиме шифрование, изменение IV приводит к разным результатам шифрования одинакового открытого текста. Значения IV не обязательно должно быть секретным, однако желательно в некоторых случаях применения.

2. Аналогично с CBC – режимом, механизм сцепления приводит к тому, что блок шифротекста c_i зависит от двух блоков открытого текста p_i и p_{i-1} . Следовательно, переупорядочивание блоков шифротекста влияет на дешифрование. Правильное дешифрование правильных предыдущих блоков шифротекста требует получения $[n/k]$ правильных предыдущих блоков шифротекста для того, чтобы реестр обратной связи содержит правильное значение.

3. Одна или больше ошибок в любом единичном r – битовом блоке шифротекста c_i влияет на дешифрование следующих $[n/k]$ блоков шифротекста, то есть пока не будет закончена обработка n битов шифротекста, после которых ошибочный блок c_i будет вытеснен из регистра обратной связи. Восстановленный открытый текст p_i' будет отличаться от p_i точно в тех позициях, в которых происходили ошибки в c_i . Другие некорректно восстановленные блоки открытого текста, обычно, будут случайными векторами, то есть будут иметь 50% ошибочных битов. В такой способ злоумышленник может осуществить

непредвиденные изменения битов p_i изменив соответственные биты в c_i .

4. CFB, как и CBC – режим, сам синхронизируется, но требует $[n/k]$ блоков шифротекста для восстановления синхронизации.

5. Для $r < n$ продуктивность режима уменьшается в n/r раз (по сравнению с CBC), потому что каждое шифрование E обеспечивает формирование только r – битов шифротекста на выходе.

Рассмотренный режим является режимом работы с r – битовыми выходными символами и r – битовой обратной связью. Этот режим похож на режим CFB, стандартизован для DES (NBS FIPS Pub 81 и ANSI X3/106) и обозначается как CFB r – битовый символ/ r – битовая обратная связь. Стандарт ISO/IES 10118:1991 определяет более общий CFB – режим. При использовании этого режима может осуществляться обработка j – битовых блоков открытого текста при r – битовой обратной связи, при чем $j \leq r$. Таким образом, для реализации режима необходимо выбрать величину обратной связи r ($1 \leq r \leq n$) и величину блока открытого текста j ($1 \leq j \leq r$).

На выход приходит k – битовый ключ K , n – битовый IV , последовательность j – битовых блоков открытого текста $P = p_1, p_2, \dots, p_u$. На выходе алгоритма формируется j – разрядные блоки шифротекста $C = c_1, c_2, \dots, c_u$.

Шифрование осуществляется таким способом.

Пре шифрованием значения вектора инициализации $I_1 = IV$ записывается в регистр обратной связи.

Для всех $1 \leq i \leq u$ выполняются такие операции:

1. Вычисляется функция шифрования:

$$O_i = E_k(I_i).$$

2. Выбирается j левых битов с вектора O_i :

$$t_i = O_i \sim j.$$

3. Формируется блок шифротекста:

$$c_i = p_i + t_i.$$

4. Формирование сменной обратной связи с блока шифротекста c_i дополнением его слева $r-j$ единицами:

$$F_i = F^{(1)}(r-j) \parallel c_i,$$

где $F^{(1)}(r-j)$ – блок с $r-j$ единиц; $\parallel$ – операция конкатенации.

5. Регистр I_i смещается на r – битов влево и записывается F_i в регистр:

$$I_{i+1} = 2^r I_i + F_i \bmod 2^n.$$

Обозначим, что для $i=u$ операции 4 и 5 не выполняются.

Расшифрование выполняется аналогично, за исключением того, что: $P_i = c_i \oplus t_i$

На рис. 2.4 приведено схематическое изображение функционирования алгоритма шифрования в CFB – режиме за ISO/IES 10118:1991.

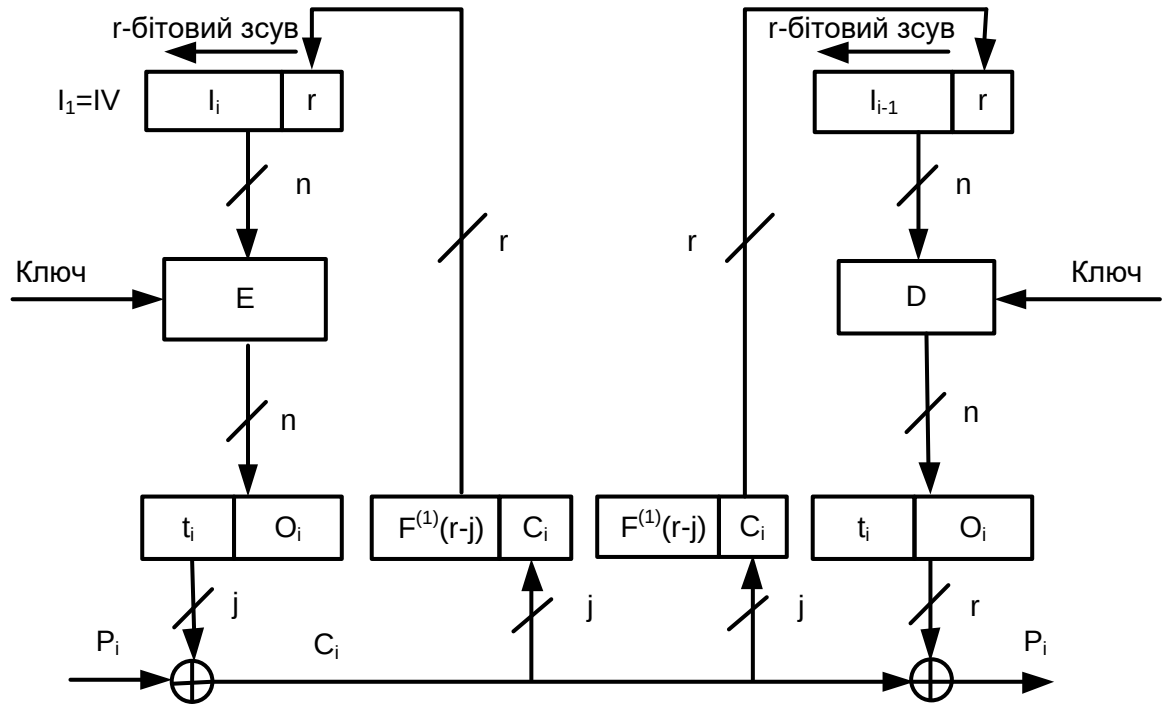


Рис. 2.4. Режим CFB – j – битовый открытый текст/ r – битовая обратная связь (j/r CFB - режим)

И, наконец то, во второй редакции стандарта ISO/IES 10116:1997 была предложена обобщенная версия CFB – режима. Стандартизованный режим обеспечивает «конвейерную обработку данных».

В ранее рассмотренных версиях CFB – режима результат шифрования одного блока открытого текста является входом для шифрования следующего блока. Это означает, что невозможны «конвейерные» вычисления, то есть невозможно приступить к шифрованию одного блока до окончания обработки предыдущего блока. Чтобы избежать этого недостатка, в новую версию CFB – режима введен уже m – разрядный буфер обратной связи, где $2n \geq m \geq n$. Также для работы необходим m – разрядный IV.

На выход алгоритма поступают: k – разрядный ключ K , m – разрядный IV, последовательность j - разрядных блоков открытого текста $P = p_1, p_2, \dots, p_i$. На выходе алгоритма формируется

последовательность j - разрядных блоков закрытой связи (регистр смещения I_i). Значение вектора инициализации $I_1=IV$.

Дальше для всех $i = \overline{1, n}$ выполняются следующие операции:

1. Выбираются n битов с левой стороны регистра смещения I :

$$X_i = I_i \sim n.$$

1. Вычисляется функция шифрования E :

$$O_i = E_k(X_i).$$

2. Выбирается j битов с левой стороны O_i :

$$t_i = O_i \sim j.$$

3. Формируется и передается блок зашифрованного текста:

$$C_i = p_i \oplus t_i.$$

4. Формируется сменная обратная связь с блока шифротекста c_i дополнением его слева $r-j$ единицами:

$$F_i = F^{(1)}(r-j) | c_i.$$

5. Смещение содержания буфера обратной связи I на r разрядов влево и запись F_i в регистр:

$$I_{i+1} = 2^r I_i + F_i \bmod 2^n.$$

Дешифрование выполняется аналогичным способом, за исключением операции 4, а именно:

$$P_i = c_i \oplus t_i.$$

На рис. 2.5 схематически приведен CFB – режим за ISO/IES 0116. Обозначим также, что во второй редакции ISO/IES 0116 рекомендуется выбирать $j=r$.

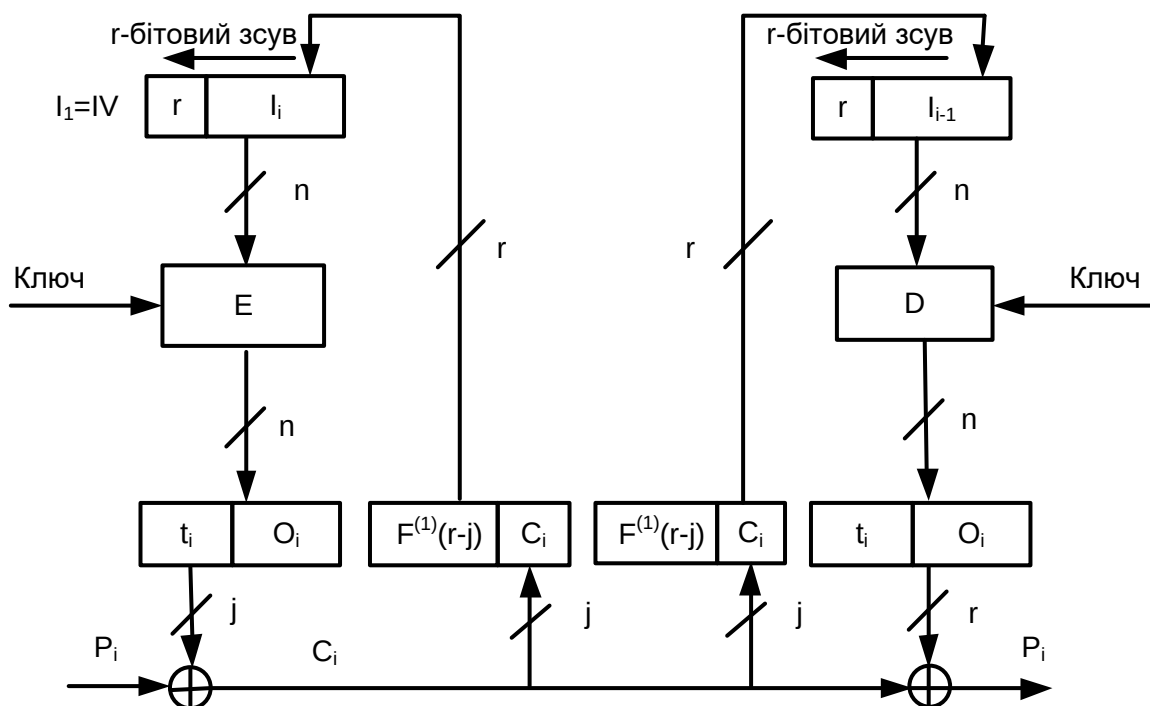


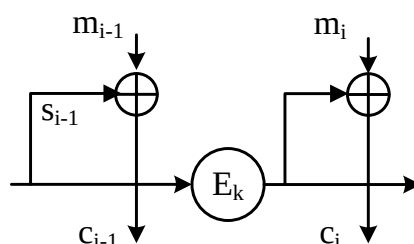
Рис. 2.5. Режим CFB за ISO/IEC 10116 с конвейерной обработкой данных.

Обратная загрузка выходных данных – OFB (Output Feedback)

В режиме OFB выходное сообщение в общем не поддается крипто – преобразованию, оно добавляется по модулю два с зашифрованными на секретном ключе s_i (s_0 является секретным параметром режима):

$$c_i = m_i \oplus s_i, \quad m_i = c_i \oplus s_i, \quad s_i = E_k(s_{i-1}).$$

В этом режиме, как и в режиме ECB, ошибки, которые могут возникнуть при передаче шифротекста по каналам связи, локализуется в блоке, не распространяясь на соседние, при чем в режиме OFB ошибочными будут только биты, которые поддавались изменению (в ECB изменится весь блок). Это дает возможность злоумышленнику незаметно для принимающей стороны подменить блок шифротекста. Возможности распараллеливания процедур шифрования/дешифрования затруднены.



Этот режим подобен CFB. Разница заключается в том, что выход алгоритма в режиме OFB подается опять в регистр смещения, тогда как

в режиме CFB в регистр смещения подается результат применения операции XOR к незашифрованному блоку и результату алгоритма.

Главное преимущество режима OFB заключается в том, что если при передаче произошла ошибка, то она не распространяется на следующие зашифрованные блоки, и тем самым сохраняет возможность дешифрования следующих блоков. Например, если появляется ошибочный бит в C_i , то это приведет только к невозможности дешифрования этого блока и получения P_i . Дальнейшая последовательность блоков будет расшифрована корректно. При использовании режима CFB C_i подается в качестве входа в регистр и является частью следующего искажения потока.

Недостаток OFB заключается в том, что он более уязвим к атакам модифицирования потока сообщений, чем CFB.

Режим OFB может быть использован для применений, у которых должно быть включено любое распространение ошибок. Режим похож на режим CFB и позволяет шифровать блоки разной длин, но как обратную связь используя не блок шифротекста, а зашифрованный блок с выхода функции E .

Распространены две версии OFB – режима работы n –разрядного блочного шифра. Больше стойкая версия ISO/IEC 10116: 1997 (ISO/IEC 10118: 1991) требует n – разрядной обратной связи (полная обратная связь) . раньше была принята версия NBS FIOS 81, который работает с $r \leq n$ разрядной обратной связью. На рис. 2.6 приведена схема OFB – режима за ISO/IEC 10116: 1997.

В этой версии OFB – режима на вход алгоритма подается: k – битовый ключ K , n – разрядный IV, последовательность r – разрядных блоков открытого текста $P = p_1, p_2, \dots, p_u, 1 \leq r \leq n \dots$ На выходе формируется последовательность r – разрядных блоков шифротекста. При дешифровании восстанавливается открытый текст.

Шифрование осуществляется таким способом.

Сначала в регистр I_i записывается значение IV. Потом для $1 \leq i \leq u$ выполняется:

1. *Вычисляется зашифрованный блок:*

$$O_i = E_k(I_i).$$

2. *С величины O_i выбираются r битов слева.*

$$t_i = O_i \sim r.$$

3. *Передают r – разрядный блок шифротекста s_i в канал связи:*

$$c_i = p_i \oplus t_i.$$

4. Восстанавливают содержание регистра смещения I_i :

$$I_{i+1} = O_i.$$

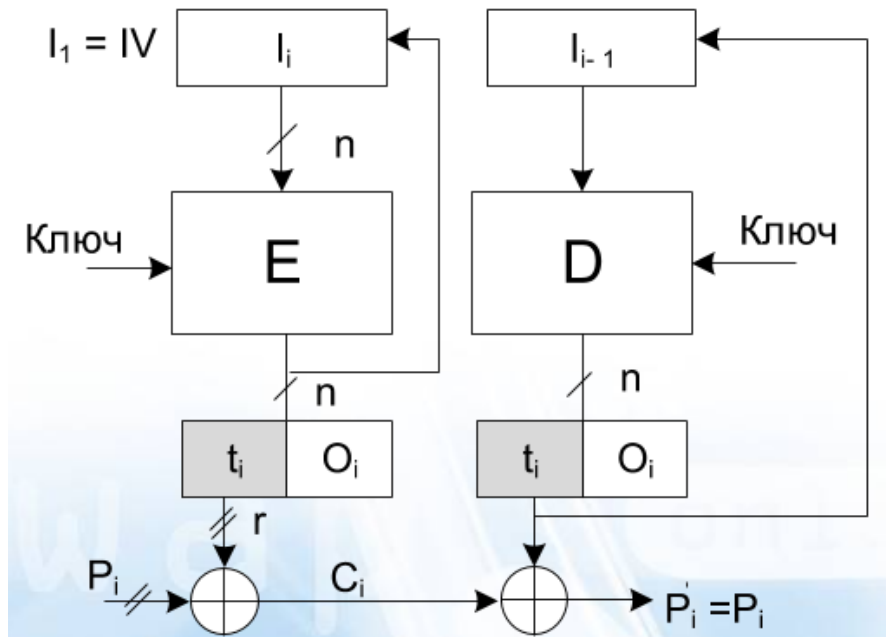


Рис. 2.6. Режим OFB r – битовый открытый текст/ n – битовая обратная связь (r/n – OFB – режим)

При расшифровании устанавливают $I_1 = IV$ и всех $1 \leq r \leq u$ осуществляют те же действия за исключением операции 3:

$$p_i = c_i \oplus t_i$$

При реализации режима OFB за NBS FIPS B1 на выход алгоритма подают: k – битовый ключ K , n – разрядный IV , последовательность r – разрядных блоков открытого текста $P = p_1, p_2, \dots, p_u$, где $1 \leq r \leq n$. На выходе формируется последовательность r – разрядных блоков шифротекста $C = c_1, c_2, \dots, c_u$.

Алгоритм шифрования и расшифрования аналогичен рассмотренным выше за исключением того, что операция восстановления регистра $I_{i+1} = O_i$ заменяется на операцию смещения регистра и записи в него значения t_i :

$$I_{i+1} = 2^r I_i + F_i \bmod 2^n.$$

Рассмотрим свойства OFB – режима:

Как и в CBC и в CFB – режимах шифрования, изменение IV приводит к разным результатам шифрования одинакового открытого текста.

Ключевой поток не зависит от открытого текста.

Одна или больше однобитовых ошибок в любом символе шифротекста c_i , влияет на дешифрование только этого символа, и

точно в тех же битовых позициях, в которых оказались ошибки в c_i . Такой способ обеспечивает полное восстановление битов открытого текста.

OFB – режим восстанавливается после ошибок в битах шифротекста, но не может сам синхронизироваться после потери битов шифротекста, которые рушат выравнивание (синхронизацию) расшифровального ключевого потока. В этих случаях необходима пересинхронизация шифра.

Для $r < n$ продуктивность уменьшается в n/r раз (как и в CFB – режиме). Однако во всех случаях, поскольку ключевой поток не зависит от открытого и шифрованного текста, он может быть исчисленный заранее чем заданный ключ и IV.

Несекретный вектор инициализации должен изменяться если в OFB – режиме повторно используется ключ K . в противном случае будет сформирован идентичный ключевой поток, и путем сложения по модулю два (XOR) соответствующих шифротекстов злоумышленник может прийти к ситуации, когда шифр будет с бесконечным ключом, где как ключ используется открытый текст.

Таким способом, выходя из свойств режима, на практике OFB – режим используется только для обеспечения конфиденциальности. В упрощенном OFB – режиме выводят восстановление входного блока как функцию счетчика, то есть: $li+1=li+1$.

Это позволяет избежать проблемы так называемого короткого цикла и обеспечить восстановление после ошибок в исчислении E . больше того, это обеспечивает свойство случайного доступа: не обязательно дешифровать i – ый блок шифротекста для того, чтобы дешифровать $i+1$ – ый блок.

Ранее версия ISO – режима была более стойкая, чем версия NBS FIPS. В OFB – режиме с полным n – разрядной обратной связью ключевой поток генерируется с использованием итеративной функции $O_i = E_k(O_i - 1)$. Поскольку E_k является перестановкой и в предположении, что k – является случайной величиной, E_k действительно является случайным выбором с множеством $(2n)!$ Перестановок за n элементами. Может быть показано, что для фиксированных (случайных) значений k и вектора инициализации ожидаемая длина цикла перед повторением любого значения O_i равняется $2n - 1$. С другой стороны, если количество битов обратной связи равняется $r < n$, как определено в FIPS 81, то ключевой поток формируется с использованием итераций $O_i = f(O_i - 1)$, где f не является функцией перестановки. В предположениях, что она ведет

себя как случайная функция, ожидаемая длина цикла составит величину порядка $2n/2$. Следовательно, лучшим является использование OFB – режима с полным n – разрядной обратной связью.

Наконец, необходимо отметить, что и OFB – режим с полной обратной связью, и режим счетчика обеспечивают возможность применения блочного шифра как генератора ключевого потока для потокового шифра. Аналогичным образом и в OFB – режиме осуществляется шифрование потока символов, а блочный шифр используется как генератор, зависимый от открытого текста.

Таблица 2.1. отображает эффект распространения ошибки в режимах шифрования.

Таблица 2.1

Эффект распространения ошибки

Режим шифрования	Эффект ошибки в бите шифротекста c_i	Эффект ошибки в i -том бите вектора инициализации IV
1	2	3
ECB	ПЛБ при розшифровании блока c_i	Не используется
CBC	ПЛБ при розшифровании блока c_i ПОБ при розшифровании блока c_{i+1}	ПОБ при розшифровании блока c_1
CFB	ПОБ при розшифровании блока c_i ПЛБ при розшифровании блоков $c_{i+1}, \dots, c_{i+n/s}$	ПЛБ при розшифровании блоков $c_1, c_2, \dots, c_i$, де $1 \leq i \leq n - i/s$.
OFB	ПОБ при розшифровании блока c_i .	ПЛБ при розшифровании блоков $c_1, c_2, \dots, c_t$.

В таблице приняты такие сокращения:

ОЛБ – ошибка в любом бите, то есть поврежденным при расшифровании с вероятностью $1/2$ может отказаться любой бит блока (или порции);

ООБ – ошибка в одном бите, то есть при расшифровании повреждается один бит, который находится в той же позиции, что и бит, который вызвал ошибку.

Алгоритмы AES

Основные параметры:

- симметричный блочный шифр с изменяемым размером входного блока: 128, 192 и 256 битов;
- длина ключа тоже изменяема, проходит те же значения;

- количество циклов обработки составляет 10, 12, 14, которая зависит от длины блока и ключа.

- AES за один этап обрабатывает весь входной блок, а не его половину, как это выполняется в фейстелевских алгоритмах.

Алгоритм AES состоит из последовательности операций, которые выполняются над промежуточным массивом 4x4 байт, который называется состояние и обозначается $S = (S_{i,j})$. В начале процесса шифрование 16 байтов состояния (128 битов) инициализируются байтами открытого текста p_i , как правило показано ниже:

$$\begin{array}{ccccccc}
 p_0 & p_4 & p_8 & p_{12} & & S_{0,0} & S_{0,1} & S_{0,2} & S_{0,3} \\
 p_1 & p_5 & p_9 & p_{13} & \xrightarrow{S=P} & S_{1,0} & S_{1,1} & S_{1,2} & S_{1,3} \\
 p_2 & p_6 & p_{10} & p_{14} & & S_{2,0} & S_{2,1} & S_{2,2} & S_{2,3} \\
 p_3 & p_7 & p_{11} & p_{15} & & S_{3,0} & S_{3,1} & S_{3,2} & S_{3,3}
 \end{array}$$

Алгоритм инициализации, как и большинство следующих операций, оптимизировано для программной реализации на 32 – битовых платформах, поскольку 4 байта каждого столбца могут обрабатываться как одно 32 – битное слово.

После инициализации и сложения циклических ключей состояние преобразовывается с помощью N_r последовательных операций раундовой функции с небольшой модификацией последнего цикла. Полный алгоритм шифрования можно описать таким псевдокодом:

```

S=AddRoundKey (P,W0)

for i=1 to Nr – 1 do

    S=SubBytes (S)

    S=ShiftRows (S)

    S=MixColumns (S)

    S=AddRoundKey (S,Wi)

endfor

S=SubBytes (S)

S=ShiftRows (S)

C=AddRoundKey (S,WNr)

```

Количество циклов N_r зависит от длины ключа и имеет фиксированные значения 10, 12, 14 для AES-128, AES-192 и AES-256

соответственно. Также заметим, что преобразование MixColumns в последнем цикле отсутствует.

Алгоритм DES

Основные параметры:

- симметричный блочный шифр;
- длина ключа – 56 бит (обычно используется 64 – битное число, но каждый восьмой бит используется для проверки парности или игнорируется);
- представляет собой 16 раундовую сбалансированную сеть Фейстеля с двумя дополнительными перестановками.

Структурную схему циклической функции алгоритма DES приведено на рис. 2.7, а схему полного алгоритма DES - на рис. 2.8.

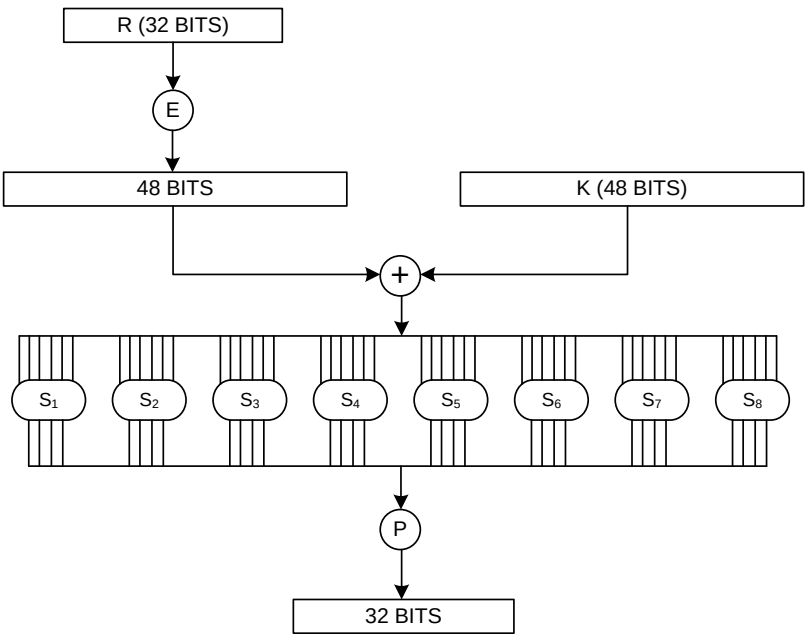


Рис. 2.7. Структурная схема циклической функции

Нелинейная замена задается таблицами замен. Например, для первого S – блока таблица замен имеет вид:

S₁

14	4	13	1	2	15	11	8	3	10	6	12	5	9	0	7
0	15	7	4	14	2	13	1	10	6	12	11	9	5	3	8
4	1	14	8	13	6	2	11	15	12	9	7	3	10	5	0
15	12	8	2	4	9	1	7	5	11	3	14	10	0	6	13

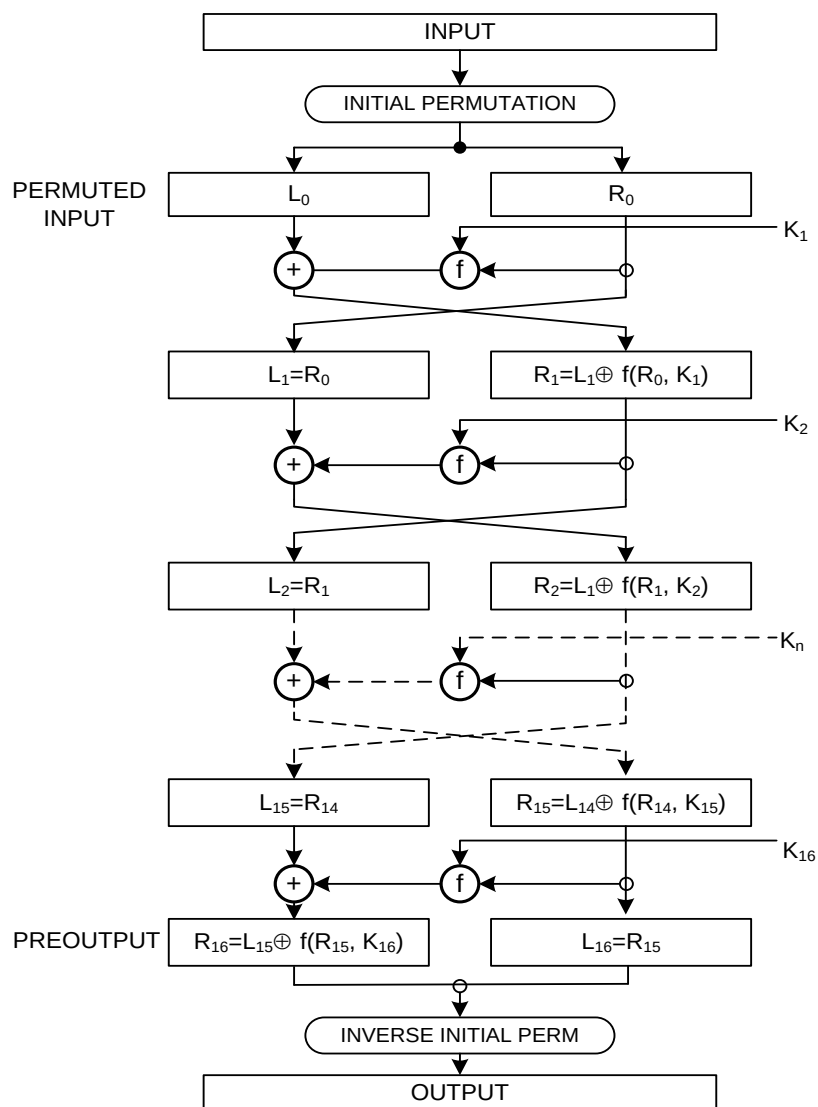


Рис. 2.8. Структурная схема алгоритма DES

Triple DES

Существует много разных вариантов тройного DES. Самыми популярными с них являются два: 3DES EDE2 (Encrypt-Decrypt-Encrypt з двома ключами) и 3DES EDE3 (Encrypt-Decrypt-Encrypt з трьома ключами).

Тройной DES с двумя ключами

В этом алгоритме используются два ключа по 56 бит, то есть общая длина ключа равняется 112 бит. Шифрование этим алгоритмом предусматривает этапы, показанные на рис. 2.9.

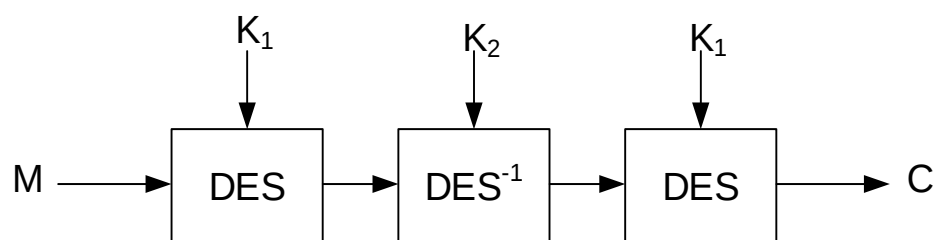


Рис. 2.9. Схема шифрования алгоритма 3DES EDE2.

Как видим на рис. 2.9, открытое сообщение M сначала шифруется обычным однократным DES на ключи K_1 , потом расшифровывается на ключи K_2 , после чего снова шифруется – на K_1 . В этом случае возрастания крипто стойкости достигается как увеличением длины общего ключа (до 112 бит + биты парности), так и количеством циклов обработки. В отличие от открытого текста. Очевидно, что тройной DES ровно в три раза медленнее за обычный, хотя и не такой медленный, как асимметричные алгоритмы. Однако быстрое развитие компьютерной техники некоторой мерой заглаживает этот недостаток. Этап расшифрования на ключи K_2 приведена для совместимости с однократным DES в случае $K_1=K_2$.

Расшифрование происходит обратным образом: на выход алгоритма подают зашифрованный текст (C), на первом этапе расшифровывают на ключи K_1 , на втором – шифруют на K_2 , на третьем – снова расшифровывают на K_1 . В результате получаем расшифрованную информацию (M).

Тройной DES с тремя ключами.

Отличие от предыдущего алгоритма в том, что тут используется три ключа шифрования, следовательно стойкость системы до атаки «грубой силой» еще возрастает. Общая длина ключа достигает $56 \cdot 3 = 168$ бит + биты парности. В случае $K_1=K_2=K_3$ 3DES EDE3 преобразовывается в однократный DES, правда, втрое медленнее.

Схема шифрования этим алгоритмом приведена на рис. 2.10.

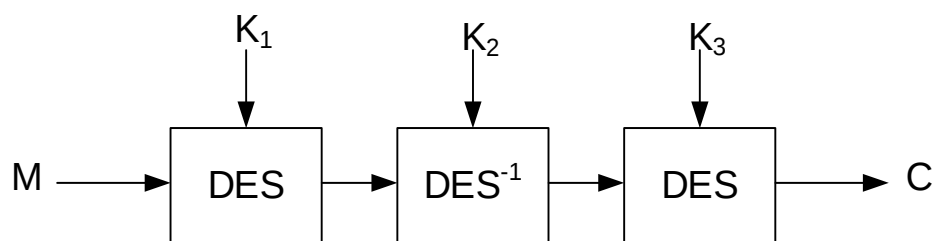


Рис. 2.10. Тройной DES с тремя ключами

Расшифрование выполняется аналогично: сначала зашифрованное сообщение расшифровывается на ключи K_3 , затем зашифровывается на ключи K_2 , и, в конце концов, снова расшифровывается, но на ключи K_1 .

Падение производительности при работе 3DES иногда очень заметное, и, например в режиме сцепления блоков это замедление не удастся компенсировать дополнительным аппаратным оборудованием. Во многих случаях, например, при шифровании критических каналов связи, такое уменьшение продуктивности недопустимо.

Алгоритм Blowfish

Алгоритм Blowfish является 16 – раундовой сеткой Фейстеля с длиной блока в 64 бита. Ключ может иметь произвольную длину в рамках 448 бит. Хотя перед началом шифрования выполняется сложная фаза инициализации, а именно шифрование данных выполняется достаточно быстро. Алгоритм предназначен, в основном для применений, где ключ меняется нечасто, к тому же существует фаза начальной аутентификации сторон, во время которой происходит согласование общих параметров шифрования. При реализации на 32 – битовых микропроцессорах с большим кэшем данных Blowfish значительно быстрее DES.

Алгоритм состоит из двух частей: развертывание ключа и шифрование данных. Развертывание ключа преобразовывает ключ длиной, по крайней мере 448 битов, в несколько массивов под ключей общей длиной 4168 байт. Каждый раунд шифра состоит из перестановки, зависящей от ключа, и замены, которая зависит и от ключа, и от данных. Раундовыми операциями является XOR и добавление 32 – битовых слов.

Blowfish использует большое количество под ключей, которые необходимо вычислить заранее, до начала процессов шифрования или расшифрования данных. Алгоритм состоит из:

1. P – массив, который содержит восемнадцать 32 – битовых под ключей: P1, P2, ..., P18.
2. Четыре 32 – битовых S – бокса с 256 входами каждый. Первый индекс означает номер S – бокса, другой индекс – номер входа:

$S_{1,0}, S_{1,1}, \dots S_{1,255},$

$S_{2,0}, S_{2,1}, \dots S_{2,255},$

$S_{3,0}, S_{3,1}, \dots S_{3,255},$

$S_{4,0}, S_{4,1}, \dots S_{4,255}.$

Метод, который используется для вычисления этих под ключей, описан далее.

Процесс шифрования

Входами алгоритма является 64 – битовый блок данных X, который делится на две 32 – битовые половины: XL и XR.

$$XL = XL \text{ XOR } P_i$$

$$XR = F(XL) \text{ XOR } XR$$

Swap XL and XR.

Структура функции F

Разделить XL на четыре 8 – битовых элемента A, B, C, D.

$F(XL) = ((S1, A + S2, B \bmod 232) \text{ XOR } S3, C) + S4, D \bmod 232.$

Расшифрование отличается от шифрования тем, что P_i используется в обратном порядке.

Генерирование под ключей

Под ключи вычисляются с использованием самого алгоритма Blowfish таким способом:

1. Инициализируется первый P – массив и четыре S – боксы с фиксированной строкой.
2. Выполняется операция XOR P1 с первыми 32 битами ключа, операцию XOR P2 с другими 32 битами ключа и т.д.. Цикл повторяется до того, пока весь P – массив не будет по битово под суммирован со всеми битами ключа. Для коротких ключей выполняется конкатенация ключа самим с собой.
3. Шифруется нулевая строка алгоритмом Blowfish, используя под ключи, описанные в пунктах (1) и (2).
P1 и P2 заменяются на результат, полученный на шаге (3).
4. Выход шага (3) шифруется, используя алгоритм Blowfish с модифицированными под ключами.
5. P3 и P4 меняются на результат, полученный на шаге (5).
6. Процесс продолжается заменой всех элементов P – массива, а потом все четыре S – боксы – результатами шифрования соответствующим образом модифицированного алгоритма Blowfish.

Для создания всех под ключей нужна 521 итерация.

Алгоритм CAST – 256

Алгоритм шифрования CAST – 256 имеет DES - подобную SPN – структуру, построенную на базе алгоритма CAST – 128, который имеет высокую стойкость к дифференциальному и линейного криптоанализа на связанных ключах. Этот шифр также имеет ряд других позитивных криптографических свойств, включая строгий лавинный критерий, критерий независимости битов (bit independence criterion – BIC), свойство дополненности и отсутствие слабых и полу слабых ключей. Он выглядит как хороший кандидат для многоцелевого использования в Интернете, где нужен криптографически сильный и доступный алгоритм.

CAST – 256 имеет размер блока 128 битов и разные длины ключей (128, 160, 192, 224 или 256 битов).

Структура алгоритму

Следующие описания CAST – 128 справедливы и для CAST – 256. CAST – 128 использует пару под ключей на раунд: 5 – битовое изменение Kr_i используется как «ротационный» ключ для i -го раунда и 32 – битовое изменение Km_i используется как «маскирующий» ключ для раунда i .

CAST – 128 использует три разных раундовых функций, как это описано ниже. Тут D – вход данной операции; I_a – самый большой байт I ; I_d – самый маленький байт I ; S_i – это i –тый S – блок, O – это выход операций). Обозначим, что «+» и «-» – это сложение и вычитание по модулю 232, «^» – побитовое сложение по модулю 2, и «<<<» – циклическая операция сдвига влево.

Тип 1	$I = ((Km_i + D) \lll Kr_i)$ $O = ((S_1I_a \wedge S_2I_b) - S_3I_c) + S_4I_d$
Тип 2	$I = ((Km_i \wedge D) \lll Kr_i)$ $O = ((S_1I_a - S_2I_b) + S_3I_c) \wedge S_4I_d$
Тип 3	$I = ((Km_i - D) \lll Kr_i)$ $O = ((S_1I_a + S_2I_b) \wedge S_3I_c) - S_4I_d$

CAST – 128 использует четыре S – блоки раундовых функций – S_1 – S_4 .

Описание CAST – 256. Пусть раундовые функции f_1 , f_2 , f_3 будут определены так же, как и в CAST – 128.

Допустим:

$BETA = (ABCD)$ будет 12 – битовым блоком, где A , B , C и D имеют длину 32 бита.

“ $BETA \leftarrow Qi(BETA)$ ” будет коротким обозначением:

$$C = C \wedge f_1(D, Kr_0(i), Km_0(i));$$

$$B = B \wedge f_2(C, Kr_1(i), Km_1(i));$$

$$A = A \wedge f_3(B, Kr_2(i), Km_2(i));$$

$$D = D \wedge f_1(A, Kr_3(i), Km_3(i)).$$

“ $BETA \leftarrow QBARI(BETA)$ ” обозначает:

$$D = D \wedge f1(A, Kr3_i), Km3_i);$$

$$A = A \wedge f3(B, Kr2_i), Km2_i);$$

$$B = B \wedge f2(C, Kr1_i), Km1_i);$$

$$C = C \wedge f1(D, Kr0_i), Km0_i);$$

(Q(*)) называется «прямая четверка - раунд» и QBAR(*) – «реверсная четверка - раунд»).

Kr_(i) = {Kr0_(i), Kr1_(i), Kr2_(i), Kr3_(i)} – множество ротационных ключей для i – й четверки – раунда, где Krj_(i) – это 5 – битовый ротационный ключ для f1, f2 или f3 (как определено выше);

Km_(i) = {Km0_(i), Km1_(i), Km2_(i), Km3_(i)} – множество маскировочных ключей для i – й четверки – раунда, где Krj_(i) - это 32 – битовый маскировочный ключ для f1, f2 или f3 (как определено выше);

KAPPA = (ABCDEFGH) – 256 – битовый блок, где A, B, ... , H имеют размер в 32 биты;

“KAPPA <- Wi(KAPPA)” обозначает:

$$G = G \wedge f1(H, Tr0_i), Tm0_i);$$

$$F = F \wedge f2(G, Tr1_i), Tm1_i);$$

$$E = E \wedge f3(F, Tr2_i), Tm2_i);$$

$$D = D \wedge f1(E, Tr3_i), Tm3_i);$$

$$C = C \wedge f2(D, Tr4_i), Tm4_i);$$

$$B = B \wedge f3(C, Tr5_i), Tm5_i);$$

$$A = A \wedge f1(B, Tr6_i), Tm6_i);$$

$$H = H \wedge f2(A, Tr7_i), Tm7_i);$$

(W(*)) называют «прямым октетом»;

“Kr_(i) <- KAPPA” обозначает:

Kr0_(i) = 5LSB(A), Kr1_(i) = 5LSB(C), Kr2_(i) = 5LSB(E), Kr3_(i) = 5LSB(G), где 5LSB(x)обозначает пять менее значимых битов x;

“Km_(i) <- KAPPA” обозначает:

$$Km0_i = H, Km1_i = F, Km2_i = D, Km3_i = B.$$

Шифрование CAST – 256.

BETA = 128 бит открытого текста.

for (i=0; i<6; i++)

..... BETA <- Qi(BETA)

.....

.....for (i=6; i<12; i++)

.....

.....BETA <- QBARi(BETA)

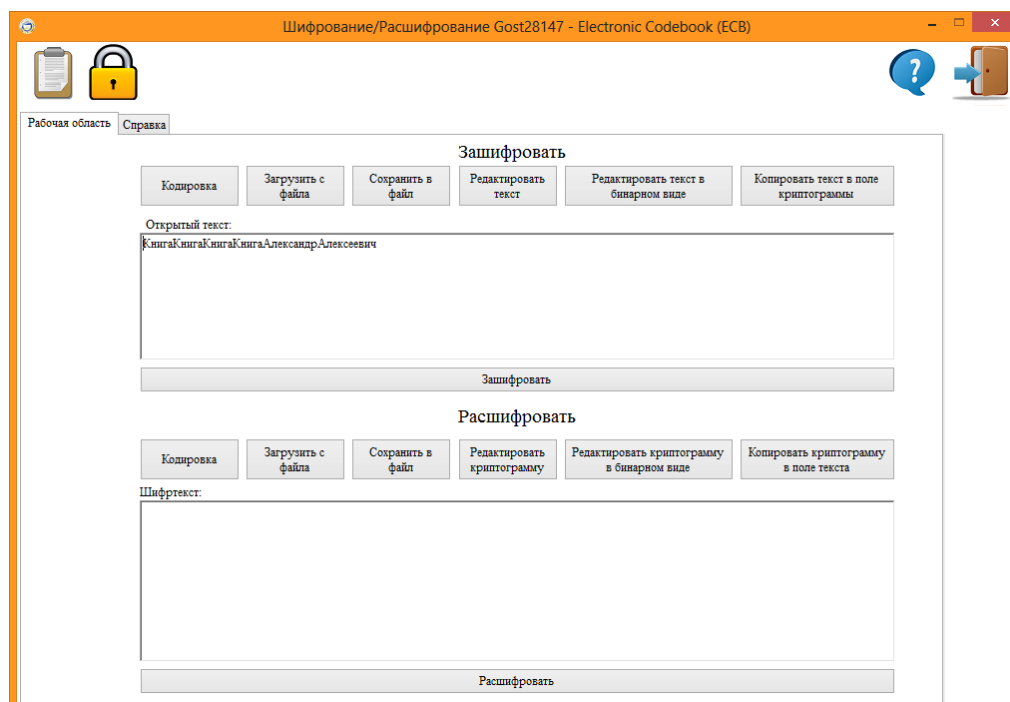


Рис. 2.16. Просмотр результата добавления открытого текста на главном экране

3. Просмотрите результат в бинарном представлении нажав на кнопку «Редактировать текст в бинарном виде»

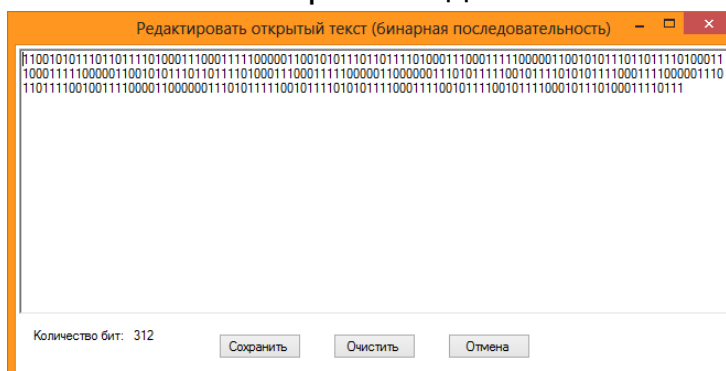


Рис. 2.17. Просмотр битового кода открытого текста

Таблица 2.2

Байты каждого слова

Слово	Книга	Книга	Книга	Книга	Александр	Алексеевич
Битовые значения в ASCII	11001010	11001010	11001010	11001010	11000000	11000000
	11101101	11101101	11101101	11101101	11101011	11101011
	11101000	11101000	11101000	11101000	11100101	11100101
	11100011	11100011	11100011	11100011	11101010	11101010
	11100000	11100000	11100000	11100000	11110001	11110001
					11100000	11100101
					11101101	11100101
					11100100	11100010
					11110000	11101000
						11110111

Длина каждого блока должна быть 8 байт (64 бита), что не соблюдены в пятом блоке.

Таблица 2.3

Байты каждого блока

Номер блока	1	2	3	4	5
Битовое значение в ASCII	11001010	11100011	11101101	11110001	11101010
	11101101	11100000	11101000	11100000	11110001
	11101000	11001010	11100011	11101101	11100101
	11100011	11101101	11100000	11100100	11100101
	11100000	11101000	11000000	11110000	11100010
	11001010	11100011	11101011	11000000	11101000
	11101101	11100000	11100101	11101011	11110111
	11101000	11001010	11101010	11100101	

4. Нажмите кнопку «Зашифровать» в главном окне. Будет предложено ввести фразу – пароль и задать шаг генератора, чтобы создать вектор инициализации.

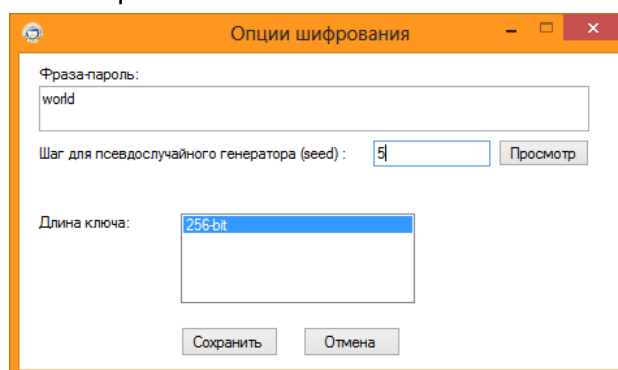


Рис. 2.18. Опции шифрования

5. Поскольку количество байтов открытого текста не кратная 8, будет выведено следующее сообщение о ошибке, рис. 2.19.

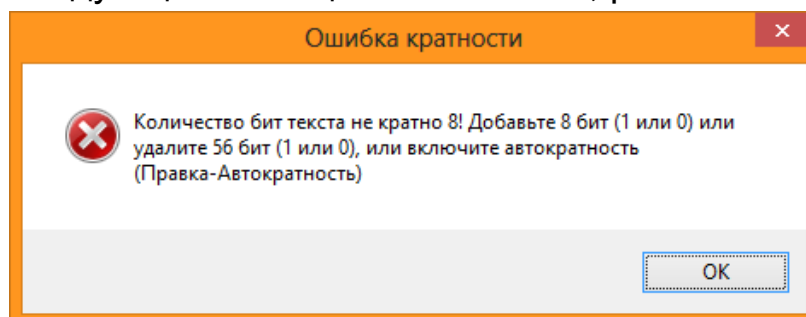


Рис. 2.19. Ошибка кратности

Решить проблему можно дописав вручную отсутствующее количество байт, или включив автоматическую кратность в настройках.

6. Решите проблему с кратностью. Один из вариантов результата решения показано в таблице 2.4.

Таблица 2.4

Сравнение последнего блока до и после редактирования

Останній блок до правки	Останній блок після правки
11100010	11100010
11101000	11101000
11110111	11110111
	11111111
	11111111
	11111111
	11111111
	11111111
	11111111

После успешного выполнения алгоритма в блоке «Расшифровать» появится зашифрованный текст, как на рис. 2.20.

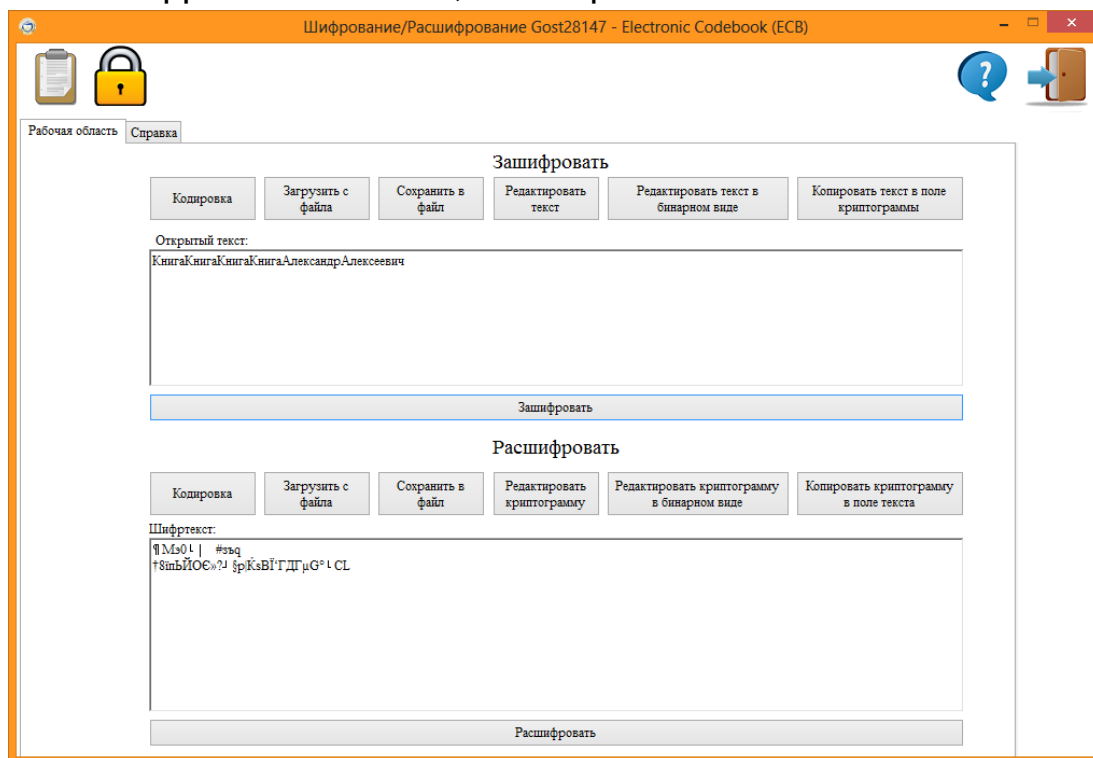


Рис. 2.20. Зашифрованный текст на главном экране

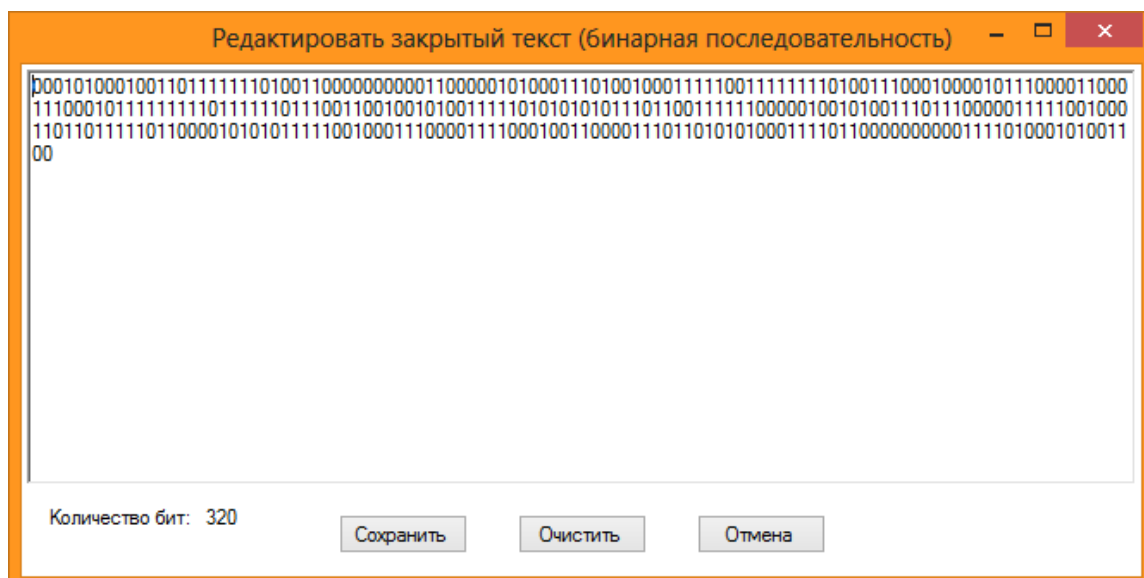


Рис. 2.21. Бинарный вариант зашифрованного текста

7. Проанализировав полученный шифротекст, сравните его с схемой используемого режима. В режиме ECB обратите внимание на повторяющиеся блоки режима.

Таблица 2.5

Сравнение открытого и зашифрованного текстов

Тип	До шифрования	После шифрования
Бинарный код	11001010111011011110100011 11000001100101011101101111 10001111100000110010101110 01000111000111110000011001 11011110100011100011111000 00011101011111001011110101 01111000001110110111100100 01100000011101011111001011 11110001111001011110010111 110100011110111	0001010001001101111111010011 0001100000101000111010010001 1111111010011100010000101110 011100010111111111011111011 1001010011111010101010111011 1000001001010011101110000011 0011011011111011000010101011 0011100001111000100110000111 010001111011000000000111101 01100
Текст	КнигаКнигаКнигаКнигаАлексан дрАлексее	Мэ0#зъq†8іпЬЮЄ»?§p ЌsВЇ'ГДГ μG°CL

8. Расшифруйте полученный шифротекст. Результат приведен на рис. 2.22.

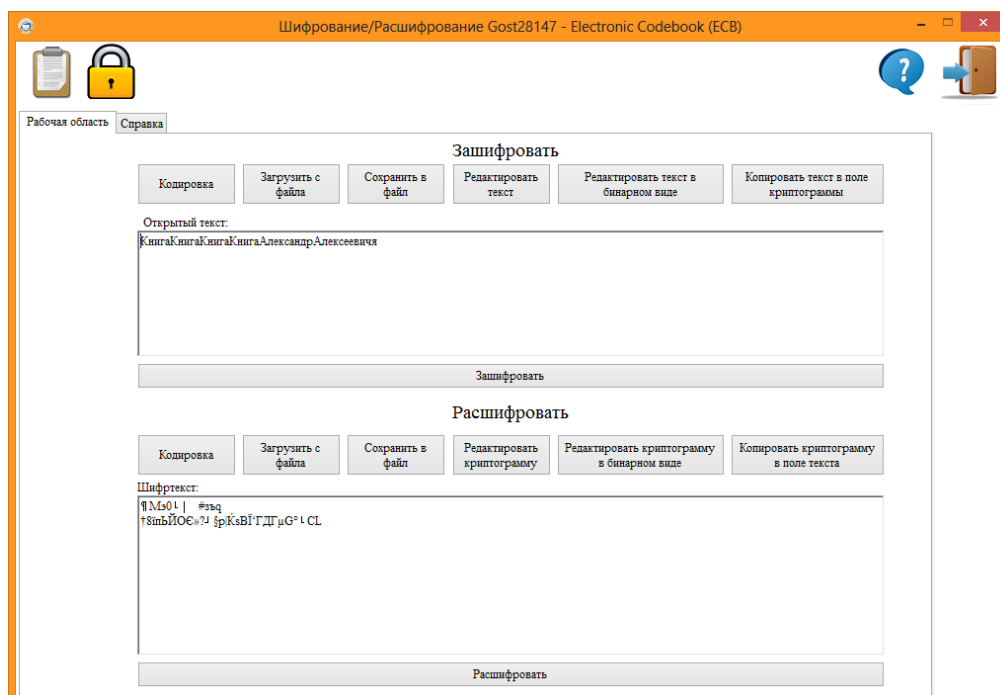


Рис. 2.22. Расшифрование шифротекста на главном экране

9. Для проведения анализа распространения ошибок внесите изменения в один бит второго блока шифротекста и расшифруйте его.

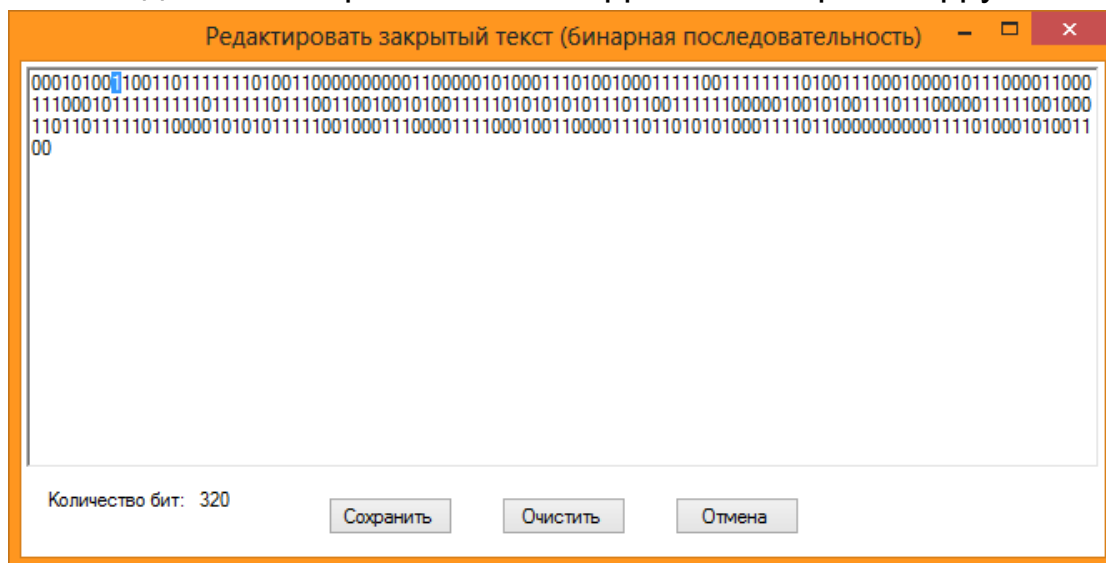


Рис. 2.23. Внесение ошибки в первый бит второго блока зашифрованного текста

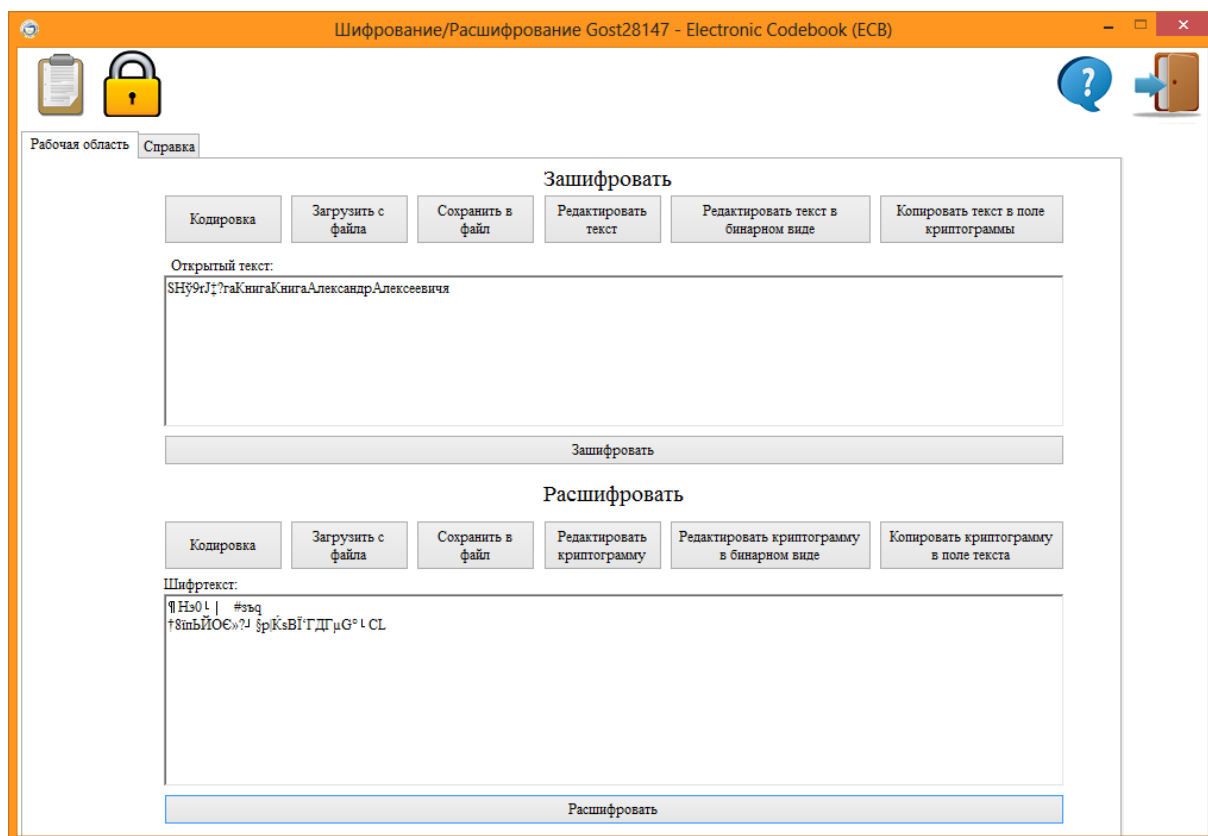


Рис. 2.24. Результат расшиврования зашифрованного текста с ошибкой

10. Проанализируйте распространение ошибки, сравните результаты со схемой режима.

Таблица 2.6

Сравнение открытого текста с ошибкой и без ошибки

Тип	Открытый текст	Расшифрование шифротекста с ошибкой в первом бите второго байта
Бинарный код	110010101110110111101000111000 11000000110010101110110111101 100011111000001100101011101101 010001110001111100000110010101 110111101000111000111110000011 000111010111110010111101010111 011110000011101101111001001111 011000000111010111110010111101 111100011110010111100101111000 110100011110111	01010011010010001010001000111001 10010101000111000011100111111111 11111000001100101011101101111010 11000111110000011001010111011011 10001110001111100000110000001110 11110010111101010111100011110000 10110111100100111100001100000011 01111100101111010101111000111100 111001011110001011110100011110111 11111
Текст	КнигаКнигаКнигаКнигаАлексан дрАлекс	SHŷ9rJ†? гаКнигаКнигаАлександрАлексеев

Выполните эти шаги для всех режимов шифрования и проанализируйте результаты.в отчет включить скриншоты открытого текста, шифротекста, шифротекста с ошибкой и открытого текста.

2.5. Задание на лабораторную работу

1. Выполните анализ режимов шифрования ECB, CTS, CBC, CFB, OFB с точки зрения распространения ошибок шифрования (см. табл. 2.4). в режимах используйте шифры соответственно варианте.

Таблица 2.4

Варианты индивидуального задания

Вариант	Шифр	Блоки для замены
1	2	3
1	Blowfish	0, 1,
	Gost	предпоследний, последний
2	Des	0, 1,
	Gost	предпоследний, последний
3	square	0, 1,
	Gost	предпоследний, последний
4	Gost	0, 1,
	rijndael	предпоследний, последний
5	Gost	0, 1,
	twofish	предпоследний, последний
6	Gost	0, 1,
	Idea	предпоследний, последний
7	Gost	0, 1,
	Mars	предпоследний, последний
8	Gost	0, 1,
	rc6	предпоследний, последний
9	Cast 256	0, 1,
	Gost	предпоследний, последний

10	RC5 Gost	0, 1, предпоследний, последний
----	-------------	-----------------------------------

i – номер по журналу;

L – длина модуля преобразования шифра в байтах;

R – количество раундов в шифре.

2.6. Контрольные вопросы

1. Какие вы знаете принципы симметричного шифрования?
2. Структура блочного симметричного шифра.
3. Какие есть уровни стойкости блочных симметричных шифров?
4. Что такое схема развертывания ключей?
5. Охарактеризуйте цепь Фейстеля, его преимущества и недостатки.
6. Какие вы знаете потоковые режимы шифрования?
7. Целостность и идентификатор сообщения.
8. Что называют набиванием сообщения?
9. Расскажите о характеристиках режимов CBC и CTS.
10. Охарактеризуйте распространение ошибки в режиме CFB.